RAJARAMBAPU INSTITUTE OF TECHNOLOGY, RAJARAMNAGAR

(An Empowered Autonomous Institute, Affiliated to Shivaji University, Kolhapur)

Industry Internship & Project (IIP) Report

on

Property Tax Management System (Salesforce Platform)

(Under "Industry Internship (II)" Track)

Submitted in partial fulfilment of the requirements for the award of

Bachelor of Technology (B. Tech)

in

Computer Science and Engineering

by

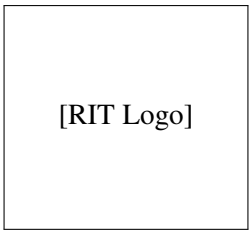
Omkar Suresh Desai
2203030

at

CodeXlabz Technologies, Atpadi, Sangli

Under the Guidance of

Mr. D. R. Patil



Department of Computer Science and Engineering

K. E. Society's

Rajarambapu Institute of Technology, Rajaramnagar

(An Empowered Autonomous Institute, Affiliated to Shivaji University, Kolhapur)

Academic Year: 2025 – 2026

K. E. Society's

Rajarambapu Institute of Technology, Rajaramnagar

(An Empowered Autonomous Institute, Affiliated to Shivaji University, Kolhapur)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Internship and Project work under Industry Internship (II) track completed at **CodeXlabz Technologies, Atpadi, Sangli** is the bona-fide work carried out by the following student of Rajarambapu Institute of Technology, Rajaramnagar during the academic year 2025–26, in partial fulfilment for the award of the degree of Bachelor of Technology (B. Tech) in Computer Science and Engineering.

The contents of this report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Name of Student: Omkar S Desai

PRN: 2203030

Date:

Place: Rajaramnagar

Mr. D. R. Patil Industry Internship & Project Mentor (College)	Mr. Ganesh Sutar Industry Internship & Project Mentor (Industry)
Name & External Examiner	
Prof. S. S. Magdum Training & Placement Coordinator	Dr. S. S. Patil Head of Department

DECLARATION

I hereby declare that this report reflects my understanding of the subject and is written in my own words. I have duly cited and referenced all the sources consulted during the preparation of this work. I confirm that this report has neither been plagiarised nor submitted for the award of any other degree or certification.

I further declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, fact, or source in this submission.

I also declare that the content of this report is original and has not been generated using Artificial Intelligence tools. Any digital or AI-based tools, if used, were limited only to minor assistance such as grammar correction or language refinement, and not for generating the core content of the report.

I understand that any violation of the above statements may lead to disciplinary action by the Institute.

Student Name	**PRN**	**Signature**
Omkar S Desai	2203030	

Date:

Place: RIT, Rajaramnagar.

CodeXlabz Technologies

Atpadi, Sangli District, Maharashtra – 415 301

CIN: [Company Registration No.] Email: info@codexlabz.com

INTERNSHIP COMPLETION CERTIFICATE

This is to certify that **Mr. Omkar S Desai**, student of Rajarambapu Institute of Technology, Rajaramnagar (PRN: 2203030) has successfully completed his Industry Internship & Project at **CodeXlabz Technologies, Atpadi, Sangli** under the "Industry Internship (II)" track for the academic year 2025–2026.

During the internship period, the student underwent structured training on the Salesforce platform covering Salesforce Administration, Apex Programming, Lightning Web Components, Experience Cloud, Agentforce and REST Integration, and subsequently designed and developed a complete **Property Tax Management System** modelled on the Greater Chennai Municipal Corporation.

The student has demonstrated diligence, technical competence, professional conduct and commitment throughout the engagement. This certificate is issued for the purpose of academic submission.

Mr. Ganesh Sutar	**Mr. Shrinath Javir**
Founder & CTO	Manager & Trainer
CodeXlabz Technologies	CodeXlabz Technologies

Date: **Place:** Atpadi, Sangli

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who contributed to the successful completion of my Industry Internship & Project (IIP) under the Industry Internship (II) track at **CodeXlabz Technologies, Atpadi, Sangli**. The journey from classroom learning to building an industry-grade software solution would not have been possible without the constant support and direction received from each of them.

I am deeply thankful to my college mentor, **Mr. D. R. Patil**, Department of Computer Science & Engineering, for her continuous guidance, patient review of every milestone, and the encouragement she extended throughout the internship and project work. Her insistence on clean, maintainable code and her stress on documentation discipline shaped the way I approached the implementation of every module. Her periodic review sessions, where she examined both the code and the design rationale behind it, instilled a practice of self-review that I now consider indispensable.

I also extend my heartfelt gratitude to **Dr. S. S. Patil**, Head of the Department of Computer Science & Engineering, for providing the necessary academic facilities, timely approvals, and a supportive learning environment that made it possible to take up an industry track of this scale. His encouragement during departmental presentations helped me articulate my work clearly and confidently.

I sincerely thank my industry mentor, **Mr. Ganesh Sutar**, Founder and CTO at CodeXlabz Technologies, Atpadi, for giving me the opportunity to work in the Salesforce CRM domain under the company's industrial training programme, and for the technical insights, code reviews and real-world delivery practices he shared despite a busy schedule. His feedback transformed my understanding of how enterprise applications are designed, tested and shipped. The weekly review meetings he conducted were instrumental in keeping the project on track and in identifying design issues early.

I would also like to thank **Mr. S. S. Magdum**, Department Training & Placement Coordinator, for his coordination, periodic follow-ups, and for being the bridge between the institute and the industry throughout my placement and internship period. My appreciation extends to all the faculty members of the Department of Computer Science & Engineering for the foundation they have built over the past four years, on which this internship work stands.

I am also grateful to the trainers at CodeXlabz Technologies, especially **Mr. Shrinath Javir**, Manager and Trainer, who delivered the structured curriculum on Salesforce administration, customisation, Lightning Web Components, integration and Agentforce, and to my fellow

interns whose collaborative discussions during workshops enriched my learning. The peer programming sessions and informal code review circles among fellow interns were particularly helpful in reinforcing concepts and discovering alternative approaches.

Finally, I express my heartfelt gratitude to my parents and family for the unwavering support and motivation they have offered during these months of intensive training, project work and report writing. Their patience and encouragement during the long working hours and weekend sessions made this achievement possible.

Mr. Omkar S Desai

PRN: 2203030

ABSTRACT

The increasing complexity of municipal property tax administration, encompassing property registration, tax assessment, payment collection, grievance handling and citizen communication, has made manual and spreadsheet-driven management both error-prone and unsustainable. Indian municipal corporations manage millions of property records across hundreds of wards and multiple administrative zones, yet many continue to rely on paper-based registration, manual assessment calculation and counter-based payment collection. This report documents an Industry Internship and Project undertaken at CodeXlabz Technologies, Atpadi, Sangli, under the company's industrial training programme, where the author was trained on the Salesforce platform and then applied that training to build a comprehensive property tax management system modelled on the Greater Chennai Municipal Corporation.

The internship phase consisted of a structured curriculum spanning approximately sixty working days, covering Salesforce administration, declarative automation through flows and approval processes, Apex programming for business logic, asynchronous processing for batch operations, Lightning Web Components for modern user interfaces, integration patterns for connecting external systems, Experience Cloud for citizen-facing portals and the Agentforce generative-AI stack for intelligent citizen assistance, supplemented by foundational training in HTML, CSS, JavaScript, JSON, data modelling and Agile delivery. Multiple assessments, hackathon phases and Mini Final Reflection Projects ensured that conceptual learning translated into demonstrable hands-on skill at each stage.

The project phase implemented a complete property tax management portal on a Salesforce Developer Edition organisation. The solution follows a citizen-centric design philosophy and is delivered across nine sequential development phases: custom object design for properties, property owners, tax assessments and tax payments with over forty custom fields; an Experience Cloud portal with electronic Know Your Customer verification using PAN and Aadhar validation; property registration with a two-level clerk-and-officer approval workflow; BSR-based tax calculation using the Greater Chennai Municipal Corporation formula with pro-rated assessment support for mid-year registrations; Razorpay payment gateway integration through a Visualforce page bridge that circumvents Lightning Locker Service restrictions on external JavaScript libraries; a two-step payment confirmation mechanism that separates transaction data persistence from approval-process-triggered status changes to avoid record-locking data loss; a real-time interactive dashboard with property cards, payment status tracking and tax overview widgets built entirely in Lightning Web Components; scheduled Apex for annual tax reassessment and due-date email reminders; monthly compounding penalty calculation; a cases and grievances module linked to Salesforce Cases with five case types, priority levels and comment threads; a zone and ward finder with Leaflet.js map integration showing two hundred wards

across four zones; and an Agentforce-powered AI assistant that answers property-tax-related queries using Knowledge Articles.

The system was tested end-to-end against the originally defined phase checkpoints. Citizens were able to register properties, complete eKYC, pay taxes through Razorpay with transaction tracking, view payment statuses on a real-time dashboard, raise grievance cases and interact with the AI assistant. Tax officers could approve property registrations and verify payments through the standard Salesforce approval process. The work demonstrates how a declarative-first, code-where-necessary approach on a low-code platform, combined with external payment gateway integration and modern generative-AI capabilities, can deliver a maintainable, configurable and extensible municipal tax management product within the time and resource constraints of a final-year industry internship.

Keywords: Salesforce, Lightning Web Components, Apex, Experience Cloud, Razorpay, Property Tax, Agentforce, Municipal Corporation, Approval Process, Payment Gateway Integration, BSR, eKYC, Sharing Sets.

CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
BSR	Basic Street Rate
CI/CD	Continuous Integration / Continuous Deployment
CRM	Customer Relationship Management
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DML	Data Manipulation Language
DOM	Document Object Model
DX	Developer Experience
eKYC	Electronic Know Your Customer
FLS	Field Level Security
FY	Financial Year
GCMC	Greater Chennai Municipal Corporation
HTML	Hyper Text Markup Language
IIP	Industry Internship & Project
JS	JavaScript
JSON	JavaScript Object Notation
LWC	Lightning Web Component
LWR	Lightning Web Runtime
MFRP	Mini Final Reflection Project
MVC	Model View Controller
OWD	Organisation-Wide Defaults
PAN	Permanent Account Number
PRN	Permanent Registration Number
PTMS	Property Tax Management System
REST	Representational State Transfer
SDK	Software Development Kit
SDLC	Software Development Life Cycle
SEO	Search Engine Optimisation
SFDC	Salesforce Dot Com
SOQL	Salesforce Object Query Language
SOSL	Salesforce Object Search Language
SQL	Structured Query Language

Abbreviation	Expansion
UIDAI	Unique Identification Authority of India
UPI	Unified Payments Interface
VF	Visualforce
VS Code	Visual Studio Code
WCAG	Web Content Accessibility Guidelines

Chapter 1

Introduction and Company Background

1.1 Introduction to the Industry Internship and Project

The Industry Internship & Project (IIP) is the capstone component of the final year of the Bachelor of Technology programme at Rajarambapu Institute of Technology, Rajaramnagar. It is designed to bridge the distance between formal classroom education and the actual practice of software engineering in industry. Under the Industry Internship (II) track, a student joins a partner organisation as a full-time intern, is trained on the technology stack and delivery practices used by that organisation, and then contributes to or builds an industry-style project. The internship and the project together form a single learning continuum: the training acquired during the internship directly feeds the project, and the project in turn consolidates the training into demonstrable, portfolio-ready competence. From the current academic year onwards, both parts are documented within one consolidated report.

The work presented in this document was carried out at CodeXlabz Technologies in Atpadi, Sangli District, Maharashtra, under the company's industrial training and internship programme. The primary technology focus of the assignment was the Salesforce platform, which is one of the most widely deployed customer relationship management (CRM) and low-code application platforms in industry today. Salesforce is used by organisations of all sizes, from small businesses to Fortune 500 enterprises, to manage customer data, automate business processes, build custom applications and expose external-facing portals for customers, partners and citizens. The training and project assignment together spanned the full duration of the IIP and produced the working property tax management system described in the chapters that follow.

The choice of the Salesforce platform was driven by several factors: the growing demand for Salesforce professionals in the Indian IT services industry, the platform's suitability for rapid application development through its declarative tools (flows, approval processes, validation rules and formula fields), the availability of Experience Cloud for building external-facing portals without requiring a separate web application stack, the maturity of the platform's security model (organisation-wide defaults, role hierarchy, sharing rules and sharing sets), and the opportunity to work with modern features such as Lightning Web Components for reactive user interfaces and Agentforce for AI-powered citizen assistance. The property tax management domain was chosen because it represents a real civic problem that affects millions of Indian citizens and is well suited to a platform-based solution involving workflow automation, role-based access control, payment gateway integration and citizen self-service.

1.2 Objectives of the Industry Internship

The objectives that were set for this internship at the outset, in consultation with both the college mentor and the industry mentor, are as follows:

- To gain a working knowledge of the Salesforce platform across its declarative and programmatic capabilities, including data modelling, security configuration, automation through flows and approval processes, Apex programming, Lightning Web Components and REST integration.
- To experience an industrial software delivery lifecycle, including requirements gathering, design documentation, code reviews, daily stand-ups, sprint planning, iterative development and milestone-based deliveries.
- To apply the platform knowledge by independently designing and building a non-trivial end-to-end application that reflects a real civic problem, in this case the management of property tax collection and citizen services for a municipal corporation.
- To gain practical exposure to the integration of an external payment gateway (Razorpay) with a Salesforce application, including the handling of test-mode transactions, callback URLs and transaction reconciliation.
- To gain exposure to the Agentforce generative-AI capabilities for citizen assistance, including agent creation, topic configuration, Knowledge Article authoring and action definition.
- To build the professional habits expected in industry: writing clean, well-documented code, maintaining design notes for non-trivial components, communicating progress clearly and working under time constraints.
- To understand the configuration of Experience Cloud portals for external citizen access, including sharing sets, guest user profiles, external identity licences and record-level security.
- To develop competence in Lightning Web Component development, including component communication patterns, lifecycle hooks, imperative versus wire-based Apex calls and navigation services.
- To prepare a comprehensive project report documenting the design decisions, implementation details, testing results, lessons learned and future scope of the developed system.

1.3 Motivation of the Present Work

Indian municipal corporations manage millions of property tax records across hundreds of wards and multiple zones. The Greater Chennai Municipal Corporation alone has over two hundred wards spread across fifteen administrative zones and is responsible for assessing and collecting property tax from hundreds of thousands of residential, commercial and government properties every financial year. A significant fraction of this administration is still managed through legacy systems, counter visits and manual record-keeping, which leads to delayed payments, incorrect assessments, poor grievance resolution and revenue leakage for the municipality.

The problems with the existing manual approach are multi-dimensional and affect both the municipality and the citizen. Property owners must physically visit municipal offices during working hours, often waiting in long queues to pay taxes that could be paid in minutes through a digital portal. This imposes a significant time and productivity cost on citizens, particularly those who are employed and cannot easily take time off for municipal errands. Tax assessments are calculated manually or through outdated software that does not account for annual rate revisions, leading to errors in the Basic Street Rate application and consequent disputes. Payment receipts are paper-based and difficult to verify or retrieve when needed for bank loans, property sales or tax exemption claims. There is no real-time visibility into payment status: after paying at the counter, a citizen must wait days or weeks for the payment to be reflected in the municipal records, with no way to track progress. Grievance resolution is slow because complaints are logged on paper and routed manually through departmental hierarchies without escalation rules or service-level tracking.

A unified, citizen-facing digital portal that allows property owners to register their properties online, complete electronic KYC verification, view tax assessments with transparent calculation breakdowns, pay taxes through a secure payment gateway with real-time transaction tracking, monitor payment statuses through an interactive dashboard, raise grievances electronically with case tracking, and get instant answers to common queries through an AI-powered assistant would significantly reduce the administrative burden on the municipal corporation, improve citizen satisfaction and increase tax collection efficiency.

The author observed that the Salesforce platform, with its mature data security model, its declarative automation through flows and approval processes, its Experience Cloud for external portals, and the recently introduced Agentforce for AI-powered citizen assistance, is well suited to such a use case. The platform's multi-tenant architecture, governor limits and built-in audit trail provide the reliability and accountability required for a government-facing application. The decision to integrate Razorpay as the payment gateway was motivated by its wide adoption in India, its support for multiple payment methods including UPI, credit cards, debit cards and netbanking, and its straightforward test-mode API that allows end-to-end payment flow testing

during development without processing real transactions. The decision to use Leaflet.js for the zone and ward finder was motivated by its open-source licence, its zero API key requirement (unlike Google Maps) and its seamless compatibility with OpenStreetMap tiles.

1.4 Company Background and Structure

1.4.1 About CodeXlabz Technologies

CodeXlabz Technologies is a specialised IT solutions company located in Atpadi, Sangli District, Maharashtra, India. The organisation provides customised software development, website designing, web application development, graphics designing, and industrial training services. Founded with the aim of bringing high-quality IT services and training to smaller cities and towns, the company has built a portfolio that spans multiple domains including education, healthcare, retail, trade and government services. The company focuses on building responsive, user-friendly, and business-oriented websites and web applications that help organisations increase their digital presence, generate business leads and improve revenue generation.

Apart from software development services, CodeXlabz Technologies provides online and offline training programmes and internship opportunities for students and professionals in future-oriented technologies. The training portfolio includes web design and development, full-stack development, artificial intelligence, machine learning, data analytics, Salesforce CRM and software development technologies. The organisation emphasises practical learning through live project implementation and industry-based training methodologies, ensuring that students gain hands-on experience with the same tools, technologies and delivery practices used in the industry. Students from Diploma, B.Tech, M.Tech, BCA, MCA and MCS programmes receive structured training with exposure to live projects and practical implementation experience. The Salesforce training track, under which this internship was conducted, is one of the newer additions to the training portfolio and reflects the growing demand for Salesforce professionals in the Indian market.

1.4.2 Mission and Vision

The company's vision statement is: "Allowing organisations to leverage technology for enterprise growth and business success." The mission statement is: "Delivering innovative and reliable solutions to meet client requirements with quality, integrity and ethical business practices." These guiding principles are reflected in the company's approach to both client projects and student training, where quality, practical relevance and ethical conduct are emphasised at every stage. The values of innovation, reliability and integrity are communicated to interns from the first day and are expected to be reflected in the quality of the code and documentation they produce.

1.4.3 Business Activities

The major business activities carried out by CodeXlabz Technologies are described below:

- **Website Designing:** The company develops attractive, responsive, and SEO-friendly websites that help businesses improve their online visibility and customer engagement. Designs are created with a focus on user experience, mobile responsiveness and cross-browser compatibility. The company has delivered websites for businesses across retail, education, healthcare and professional services sectors.
- **Web Development:** The organisation builds robust and scalable web applications focusing on both frontend and backend technologies to ensure performance, usability and reliability. Technologies used include HTML5, CSS3, JavaScript, React.js, Node.js, Python, PHP and various database systems including MySQL and MongoDB. Full-stack development services cover the entire application lifecycle from requirements gathering through deployment and maintenance.
- **Graphics Designing:** Professional graphic designing services are offered including user interface design, motion graphics, brand visual development, interactive web design, logo creation, brochure design and social media graphics. These services help organisations establish a strong visual identity and maintain consistency across their digital and print communications.
- **Training and Internship Programmes:** Industry-oriented training and internship programmes for students, providing exposure to live projects, modern development tools, version control systems, Agile delivery practices and industry best practices. The training is structured into modules with assessments at each stage, and culminates in a project phase where students build a working application that demonstrates their acquired skills.

1.4.4 Products Delivered by the Organisation

The company has developed and delivered multiple software products and solutions across different domains:

Table 1.1: Products delivered by CodeXlabz Technologies

No.	Product	Description
1	E-Commerce Systems	Online retail platforms with product catalogues, shopping carts, payment processing, order tracking and inventory management.

No.	Product	Description
2	College IDPro Management	Student and staff identity management with ID card generation, barcode scanning, attendance tracking and administrative dashboards.
3	Hospital Management System	Patient registration, appointment scheduling, doctor assignment, billing, pharmacy management and inventory control for healthcare facilities.
4	Online Examination Portal	Web-based examination system with question banks, randomised question selection, timed tests, automated grading and result analytics.
5	Static Websites	Informational websites for businesses with responsive design, SEO optimisation, contact forms and map integration.
6	Dynamic Websites	Database-driven websites with user authentication, content management systems, admin panels and interactive features.
7	Field Service Assistance App	Mobile-friendly applications for field service teams with GPS-based task assignment, checklist management, photo capture and reporting.
8	Blogging Websites	Content management platforms for bloggers and content creators with post scheduling, category management and analytics.
9	Import-Export Business Portal	Trade management portal with documentation workflows, shipment tracking, compliance checklists and partner communication tools.

This breadth of delivery experience across different domains provided the company with a strong foundation for taking on the Salesforce-based property tax management system undertaken during this internship, particularly in the areas of portal development, payment integration and workflow automation.

1.4.5 Organisational Structure

CodeXlabz Technologies follows a decentralised organisational structure where employees are empowered with decision-making authority and encouraged to solve challenges creatively rather than waiting for top-down directives. The organisation consists of approximately nine team members including management and technical staff. The key personnel are listed in the following table:

Table 1.2: Key personnel at CodeXlabz Technologies

Name	Designation	Role and Responsibilities
Mr. Ganesh Sutar	Founder, CTO	Overall technical direction, architecture decisions, client relationship management, intern mentorship and code reviews
Miss Tanuja Sutar	Finance Manager	Financial operations, budgeting, invoicing, compliance and administrative coordination
Mr. Shrinath Javir	Manager, Trainer	Training curriculum design and delivery, daily progress tracking, intern assessment and project supervision
Technical Team (5)	Developers	Software development, testing, deployment, client support and peer mentoring for interns

The flat organisational structure facilitates direct communication between interns and senior technical staff, which proved valuable during this project when rapid clarification on Salesforce platform behaviour or design trade-offs was needed.

1.4.6 Infrastructure Facilities

The company provides modern infrastructure facilities to support both client projects and training activities:

- Twenty-four-hour laboratory facility with dedicated workstations for each intern, ensuring uninterrupted access to the development environment.
- High-speed internet connectivity essential for cloud-based Salesforce development, which requires continuous access to the Salesforce Developer Edition organisation hosted on Salesforce's cloud infrastructure.
- Battery backup systems (UPS) to ensure uninterrupted work during power fluctuations, which are common in smaller towns.
- Wi-Fi campus for mobile device connectivity, enabling testing of the Experience Cloud portal on mobile browsers.
- Structured cabling infrastructure for reliable wired networking between workstations.
- Library facility with technical books, reference materials and access to online learning platforms.
- Well-equipped computer laboratories with modern hardware running Windows and Linux operating systems.

- Dedicated lecture rooms for training sessions, presentations, code walkthroughs and sprint review meetings.
- Soft-skill development programmes covering communication, presentation, teamwork and professional etiquette.
- Interview preparation sessions and mock interview rounds to prepare interns for industry placements.
- Online aptitude test facility for periodic assessment of technical and analytical skills.

The availability of these facilities, particularly the round-the-clock laboratory access and reliable high-speed internet connectivity, was essential for the Salesforce development work which required continuous access to the cloud-based Salesforce Developer Edition organisation for building, testing and debugging the property tax management system.

1.4.7 Structure of the Engagement

For an intern at CodeXlabz Technologies, the reporting structure includes the founder and CTO who provides overall technical direction and conducts periodic architecture reviews and code walkthroughs, a designated trainer who delivers the day-to-day curriculum and monitors progress through assessments and check-ins, and technical team members who assist with troubleshooting, provide peer-level guidance and share practical tips from their project delivery experience. The author's industry mentor, Mr. Ganesh Sutar, served as the primary point of guidance for both the technical content of the project and the professional expectations of the engagement. Weekly progress meetings were held every Friday afternoon to review completed work, examine code quality, identify blockers, discuss design alternatives and plan the next set of deliverables. These meetings followed a structured format similar to an Agile sprint review, with a demonstration of working features followed by a feedback and planning discussion.

1.5 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 describes the internship training phase in detail, including the module-wise curriculum on the Salesforce platform, the assessments and hackathons conducted, and the key learnings carried into the project. Chapter 3 presents a literature survey of the problem domain, examining the current state of property tax administration in India and the digital initiatives undertaken by various municipal corporations. Chapter 4 captures the system analysis, including stakeholder identification, functional and non-functional requirements and the technology choices made. Chapter 5 documents the system design, including the data model, security architecture, module decomposition and integration patterns. Chapter 6 presents the implementation in nine sequential phases corresponding to the development plan executed during the project. Chapter 7 describes the testing methodology and

the results obtained. Chapter 8 concludes the report with a summary of contributions, the limitations of the current implementation and the scope for future enhancements. The references and appendices follow at the end.

Chapter 2

Weekly Jobs Summary

2.1 Overview

This chapter presents a week-by-week summary of all activities, tasks and projects handled during the twenty-one week Industry Internship at CodeXlabz Technologies, Atpadi, Sangli, from 5 January 2026 to 31 May 2026. The internship comprised two phases: a structured Salesforce platform training phase spanning the first sixteen weeks (Weeks 1–16), and a project development phase spanning the final five weeks (Weeks 17–21). All activities are summarised in tabular form below.

Table 2.1: Weekly Jobs Summary – Industry Internship at CodeXlabz Technologies

Week No.	Duration	Activities / Tasks / Projects Handled
Week 1	05/01/2026 – 11/01/2026	Completed joining formalities and onboarding process. Attended orientation sessions covering company policies, workplace practices and team introductions. Installed and configured the development environment: Visual Studio Code, Salesforce CLI, Salesforce Extension Pack and Git. Provisioned personal Salesforce Developer Edition organisation. Began Module 1: Web Foundations – HTML5 structural and semantic elements, CSS3 selectors, box model and flexbox layouts.
Week 2	12/01/2026 – 18/01/2026	Continued Module 1: CSS3 grid layouts, responsive design with media queries, JavaScript ES6 features (let, const, arrow functions, template literals, destructuring, promises and async/await), Document Object Model and event handling. Studied JSON as the data interchange format for Salesforce REST APIs. Completed Module 1 hands-on exercises and assessment.

Week No.	Duration	Activities / Tasks / Projects Handled
Week 3	19/01/2026 – 25/01/2026	Began Module 2: Salesforce Administration Fundamentals. Topics covered: multi-tenant cloud architecture and governor limits, Salesforce org structure, user management and licence types, difference between profiles and permission sets. Hands-on exercises in user creation, profile assignment and permission set configuration within Developer Edition org.
Week 4	26/01/2026 – 01/02/2026	Continued Module 2: role hierarchy and its effect on record visibility, organisation-wide defaults (OWD), sharing rules, sharing sets for community users, public groups, record types and page layouts. Completed exercises observing record-level security changes. Completed Module 2 assessment. Commenced MFRP 1: Contact Management Application (data modelling, validation rules, basic reporting).
Week 5	02/02/2026 – 08/02/2026	Completed and submitted MFRP 1. Began Module 3: Data Modelling and Object Relationships. Topics covered: standard Salesforce objects (Account, Contact, Opportunity, Case, Lead), custom object creation, all supported custom field data types (text, number, currency, date, picklist, multi-select picklist, checkbox, formula, roll-up summary, geolocation and external lookup). Explored lookup vs. master-detail relationship behaviour.
Week 6	09/02/2026 – 15/02/2026	Continued Module 3: cascade-delete behaviour in master-detail relationships, inheritance of sharing settings, roll-up summary fields on master-detail parents, junction objects for many-to-many relationships and Schema Builder visualisation. Built a library management system data model as the module assessment exercise. Completed Module 3 assessment.
Week 7	16/02/2026 – 22/02/2026	Began Module 4: Declarative Automation. Topics covered: validation rules, formula fields, legacy workflow rules and Process Builder (presented as deprecated). Introduced Flow Builder and its variants: screen flows for guided user interactions, record-triggered flows, scheduled flows and autolaunched flows. Completed multiple screen flow and record-triggered flow exercises.

Week No.	Duration	Activities / Tasks / Projects Handled
Week 8	23/02/2026 – 01/03/2026	Continued Module 4: approval processes – entry criteria, multi-step approval routing (specific user, queue, manager), final approval and rejection actions, email templates for notifications and recall behaviour. Completed Hackathon 1 (Saturday): built a complete leave management system using only declarative tools (custom objects, validation rules, approval process, screen flow, record-triggered flow, Lightning App Builder page) within six hours. Commenced MFRP 2: Expense Approval System.
Week 9	02/03/2026 – 08/03/2026	Completed and submitted MFRP 2. Began Module 5: Apex Programming. Topics covered: Apex syntax and comparison with Java, primitive data types, sObject type, collections (List, Set, Map), control flow constructs, class definition, methods, inheritance, interfaces, abstract classes and access modifiers. Studied with-sharing, without-sharing and inherited-sharing keywords.
Week 10	09/03/2026 – 15/03/2026	Continued Module 5: DML statements (insert, update, upsert, delete, undelete, merge), Database methods for partial success, bulkification principle, trigger frameworks with handler class delegation. Covered SOQL in detail: filter clauses, sorting, aggregate queries with GROUP BY and HAVING, semi-joins, anti-joins and parent-to-child and child-to-parent relationship queries. Introduced SOSL for cross-object full-text searching. Completed Module 5 assessment.
Week 11	16/03/2026 – 22/03/2026	Began Module 6: Asynchronous Apex. Topics covered: future methods, queueable Apex (chainable jobs), batch Apex with scoped start/execute/finish pattern, schedulable Apex for time-based recurring jobs, callout rules for asynchronous contexts and elevated governor limits in async execution. Implemented a combined schedulable–batch pattern. Studied MFRP 3 requirements.

Week No.	Duration	Activities / Tasks / Projects Handled
Week 12	23/03/2026 – 29/03/2026	Began Module 7: UI Development Frameworks. Covered Visualforce briefly: tag-based markup, controllers, view state and standard set controllers. Transitioned to Lightning Web Components (LWC): LWC bundle structure (HTML, JS, metadata XML), use of decorators (`@api`, `@track`, `@wire`), lifecycle hooks (constructor, connectedCallback, renderedCallback, disconnectedCallback) and Lightning Data Service via `@wire`.
Week 13	30/03/2026 – 05/04/2026	Continued Module 7: LWC event handling (DOM events and custom events), Navigation Service, Toast Notification Service and base Lightning component library (lightning-input, lightning-button, lightning-card, lightning-datatable, etc.). Built a custom data table with inline editing, sortable columns and search. Completed Hackathon 2 (Saturday): built a real-time notification system using LWC and Platform Events within six hours. Commenced MFRP 3: e-Learning Portal on Experience Cloud.
Week 14	06/04/2026 – 12/04/2026	Completed and submitted MFRP 3. Began Module 8: Integration with External Systems. Topics covered: inbound integration via Apex REST (`@RestResource`) and SOAP services; outbound integration via HTTP callouts using Http, HttpRequest and HttpResponse classes; named credentials vs. remote site settings for endpoint management; authentication patterns (Basic Auth, OAuth 2.0 web server flow, JWT bearer flow, client credentials flow) and Connected Apps configuration.
Week 15	13/04/2026 – 19/04/2026	Continued Module 8: consumed a public weather REST API and exposed a custom Apex REST endpoint returning JSON. Began Module 9: Experience Cloud – site creation using Customer Service template, guest user access configuration, licence types (Customer Community, Customer Community Plus, Partner Community), sharing-set mechanism, published-versus-draft model and LWR sites. Built a small partner portal as hands-on exercise.

Week No.	Duration	Activities / Tasks / Projects Handled
Week 16	20/04/2026 – 26/04/2026	Completed Module 9 assessment. Module 10: Reports and Dashboards – report types, four standard report formats (tabular, summary, matrix, joined), bucket fields, conditional highlighting, cross-filters, dashboard chart types and dynamic dashboards. Module 11: Agentforce and Generative AI – agent topics, natural-language instructions, actions (flows, Apex, prompt templates), Knowledge Articles, Einstein Trust Layer configuration and a simple FAQ agent exercise. Training phase concluded. Began project planning and requirements review.
Week 17	27/04/2026 – 03/05/2026	**Project Phase 1 – Custom Object Design.** Created Property__c, Property_Owner__c, Tax_Assessment__c and Tax_Payment__c objects with all custom fields (over forty total). Configured validation rules for data integrity (PAN format regex, non-zero plinth area, Razorpay ID presence). Enabled field history tracking on status, monetary and structural fields. Reviewed object relationships and OWD settings.
Week 18	04/05/2026 – 10/05/2026	**Project Phase 2 – Experience Cloud Portal and eKYC.** Created and activated the Experience Cloud site (`/property-tax`). Configured guest user access, sharing sets and Customer Community Plus Login profile. Built the eKYC LWC component with PAN regex check (Apex stub callout), Aadhar Verhoeff checksum validation (client-side JavaScript) and OTP generation/verification via Apex. **Project Phase 3 – Property Registration and Approval Workflow.** Built the property registration screen flow (three-screen address–structure–owner form). Configured two-step approval process routing to clerk queue and zone officer with thirty-day recall. Wrote Apex trigger on Property__c for assessment creation on final approval.

Week No.	Duration	Activities / Tasks / Projects Handled
Week 19	11/05/2026 – 17/05/2026	**Project Phase 4 – BSR-Based Tax Calculation.** Implemented TaxAssessmentService Apex class: BSR lookup from Ward_BSR__mdt custom metadata, four-factor multiplication (usage, age, structure, twelve months), library and sanitation cess application, pro-rated calculation for mid-year registrations. Wrote TaxAssessmentServiceTest with seven scenarios achieving 96% coverage. **Project Phase 5 – Razorpay Payment Gateway Integration.** Built Visualforce page bridge for Razorpay JavaScript SDK (Locker Service workaround). Implemented Apex callout to Razorpay Orders API. Configured HMAC-SHA256 signature verification using Named Credential. Integrated VF page as iframe in LWC modal.
Week 20	18/05/2026 – 24/05/2026	**Project Phase 6 – Two-Step Payment Confirmation.** Diagnosed and resolved the record-locking data-loss bug. Implemented PaymentVerificationBatch scheduled job (runs every five minutes) to transition Pending payments to Verified after parent approval completion. **Project Phase 7 – Real-Time Citizen Dashboard.** Built citizenDashboard LWC with propertyCard, paymentStatusBadge and taxOverviewWidget child components. Configured `@wire` subscription with Lightning Data Service cache invalidation for real-time refresh. **Project Phase 8 – Scheduled Apex for Reassessment and Reminders.** Deployed AnnualReassessmentSchedule + Batch (April 1 at 00:01 IST) and DueDateReminderSchedule + Batch (daily at 09:00 IST). Implemented tiered reminder email templates. Configured penalty flow (2% per month, capped at 24%).

Week No.	Duration	Activities / Tasks / Projects Handled
Week 21	25/05/2026 – 31/05/2026	**Project Phase 9 – Grievances, Zone Finder and Agentforce.** Configured Case object with Property_Grievance record type (five case types, three priority levels) and zone-based assignment rules. Built zoneFinder LWC with Leaflet.js static resource, Ward_Geometry__mdt GeoJSON polygons and geocoding. Configured Agentforce agent with three actions and fifteen Knowledge Articles. Conducted end-to-end integration testing across all five scenarios. All twenty UAT test cases passed. Prepared final report documentation and submitted deliverables.

Chapter 3

Literature Survey and Problem Domain

3.1 Property Tax Administration in India

Property tax is a recurring tax levied annually by urban local bodies on property owners within their jurisdiction. It is one of the most significant sources of own-source revenue for Indian municipal corporations and is governed primarily by the respective state municipal acts, supplemented by ward-level by-laws. The Constitution of India, through entries forty-nine and fifty of the State List, places taxes on lands and buildings within the domain of state governments, which in turn delegate the collection authority to municipal corporations, municipal councils and nagar panchayats under the seventy-fourth constitutional amendment.

The methodology of property tax assessment varies across municipalities but typically falls into one of three broad approaches. The first is the Annual Rental Value (ARV) system, which assesses tax based on the notional rent that the property could fetch in the open market. The second is the Capital Value System, which assesses tax based on a percentage of the market value of the property as captured in the state's ready-reckoner. The third is the Unit Area Value (UAV) system, which assigns a per-unit-area rate (typically per square foot or square metre) to each street or zone, multiplied by the built-up area of the property, with adjustments for usage type (residential, commercial, industrial), age of the building and structural type. The UAV system is the dominant approach in many South Indian cities including Chennai, where the per-unit rate is called the Basic Street Rate.

3.2 Greater Chennai Municipal Corporation as a Reference

The Greater Chennai Municipal Corporation (GCMC) was chosen as the reference municipality for this project because it has one of the largest and most well-documented property tax administrations in India. The corporation administers a city of approximately seven million residents across two hundred wards distributed over fifteen administrative zones. For the purposes of this project, a simplified model with four zones and two hundred wards was adopted to reduce data preparation effort without losing the architectural characteristics of the actual system. The GCMC formula for property tax under the Unit Area Value system is approximately:

$$\textit{Annual Property Tax} = \textit{Plinth Area} \times \textit{Basic Street Rate (BSR) per sq.ft} \times \textit{Usage Factor} \times \textit{Age Factor} \times \textit{Structure Factor} \times 12 \text{ months}$$

with subsequent additions of library cess (typically ten per cent), sanitation cess, education cess and other smaller statutory levies as applicable. The Basic Street Rate is revised

periodically by the corporation in consultation with the state government and is published in the official gazette. The simplified model used in this project applies a uniform set of factors but preserves the structure of the calculation so that the actual factors can be substituted later without a code change.

3.3 Existing Digital Initiatives

Several Indian municipal corporations have launched digital property tax portals in the last decade. Notable examples include the Brihanmumbai Municipal Corporation portal, the Pune Municipal Corporation portal, the Bangalore Bruhat Bengaluru Mahanagara Palike portal and the Greater Chennai Corporation portal itself. These portals typically allow citizens to look up their property by an existing assessment number, view the current outstanding tax and pay through an integrated payment gateway. However, several limitations are common across these existing systems:

- **Limited self-registration.** Most portals require an existing assessment number issued by the municipality. New properties cannot be registered through the portal; the citizen must visit the municipal office for the initial registration. This perpetuates the counter-visit dependency that the portal was meant to eliminate.
- **Opaque calculations.** Citizens see the final tax amount but not the inputs to the calculation. There is no breakdown of plinth area, BSR, usage factor and other multipliers, which makes it impossible to verify the assessment or to raise an informed dispute.
- **No real-time payment status.** After payment, the receipt is generated, but if the payment fails or is in a pending state at the gateway, the citizen often cannot see the current status without contacting the municipality.
- **Weak grievance integration.** Grievance redressal is typically handled through a separate complaints portal or a generic municipal grievance system that is not linked to the citizen's property records, making it difficult to provide context-aware support.
- **No conversational interface.** Citizens with simple questions ("what is my next due date?", "how is the BSR for my street calculated?") must either find the answer in a long FAQ page or call a helpline. There is no chat-style assistance.

The system designed in this project addresses each of these limitations as part of its core feature set, taking advantage of the platform capabilities of Salesforce to deliver the missing functionality.

3.4 Salesforce as a Government Platform

Salesforce has been adopted as a platform for government applications across several countries. The Salesforce Public Sector Solutions product line offers pre-built data models and accelerators for grants management, licensing and permitting, inspections, case management and benefit eligibility determination. While the present project did not use the Public Sector Solutions product (which requires a separate licence not available on a Developer Edition organisation), it draws inspiration from the design patterns documented in the public Salesforce architecture guides, particularly the patterns around citizen self-service portals, document submission with verification workflows and integration with external payment and identity verification services.

3.5 Gap Analysis

The literature survey identified a clear gap that the present project aims to fill: an end-to-end citizen-facing property tax portal that supports self-registration with eKYC, transparent BSR-based assessment, secure payment integration with real-time status, integrated grievance management linked to property records and AI-assisted citizen support. While individual elements of this combination exist in various deployed systems, the unified delivery on a single platform with declarative configuration and minimal custom code is the distinctive contribution of this project.

Chapter 4

System Requirements and Analysis

4.1 Stakeholder Identification

The system serves three primary stakeholder categories, each with distinct needs and access requirements:

- **Citizens (Property Owners):** The primary external users. They register their properties, complete eKYC, view assessments, pay taxes, monitor payment status, raise grievances and interact with the AI assistant. They access the system through the Experience Cloud portal as Customer Community Plus Login users.
- **Tax Clerks:** Internal users who perform the first level of review on property registrations. They verify the citizen-supplied details against supporting documents, request clarifications if needed and forward complete applications to the tax officer. They are also the first point of contact for grievance triage. They access the system through the standard Salesforce Lightning Experience.
- **Tax Officers:** Internal users who perform the second-level approval on property registrations, oversee escalated grievances, monitor payment collection and approve payment confirmations. They have broader visibility across all properties in their assigned zones.
- **System Administrator:** A platform-level user responsible for org configuration, user management, security maintenance and routine operational tasks such as the configuration of scheduled jobs and the publication of Knowledge Articles for the AI assistant.

4.2 Functional Requirements

The functional requirements were grouped under the same nine phases that structure the implementation:

Table 4.1: Functional requirements by phase

FR	Capability	Description
1	Property Registration	Citizens shall be able to register a new property, providing structural details (plinth area, usage type, structure type, age) and a complete address with zone and ward selection.

FR	Capability	Description
2	eKYC Verification	The system shall verify the citizen's identity through PAN format validation, Aadhar checksum validation (Verhoeff algorithm) and OTP-based mobile verification simulated via Apex.
3	Two-level Approval	Every property registration shall pass through a clerk-level review and an officer-level approval before the property is activated and assessed.
4	Tax Calculation	The system shall calculate the annual tax for each property using the GCMC formula, with pro-rated calculation for properties registered after the start of the financial year.
5	Payment Gateway	The system shall accept payments via Razorpay test mode supporting UPI, cards and netbanking, with real-time status reflection.
6	Payment Confirmation	Successful payments shall be confirmed in two steps: a Payment Pending record created at gateway success, and a Payment Verified status set after the approval-based reconciliation.
7	Citizen Dashboard	Citizens shall see a real-time dashboard with cards for each owned property, payment status indicators, due-date alerts and quick-action buttons.
8	Annual Reassessment	The system shall automatically re-run the tax assessment for every active property at the start of each financial year via a scheduled job.
9	Reminders	The system shall send email reminders fifteen days before, seven days before and on the due date for each unpaid assessment.
10	Grievance Management	Citizens shall be able to raise grievance cases linked to a property, choosing from five case types (assessment dispute, payment issue, refund request, ownership change, other) with three priority levels.
11	Zone and Ward Finder	Citizens shall be able to identify their zone and ward through an interactive map with two hundred ward polygons across four zones.
12	AI Assistant	Citizens shall be able to ask questions through an Agentforce-powered conversational assistant that cites Knowledge Articles.

4.3 Non-Functional Requirements

- **Security.** All citizen data shall be protected through organisation-wide defaults set to Private for the Property and Tax Payment objects. Sharing sets shall grant each citizen access only to their own records. PAN and Aadhar numbers shall be stored in encrypted custom fields where available, and shall never be exposed through the Experience Cloud portal beyond the last four digits.
- **Performance.** Page load times for the citizen dashboard shall remain under two seconds for a citizen with up to ten properties. Apex transactions shall remain within the governor limits with significant headroom (no more than fifty per cent of any limit).
- **Scalability.** The data model and code shall be capable of handling at least one hundred thousand property records and one million payment records without redesign. Batch jobs shall use scoped batch sizes (typically two hundred) to remain within limits.
- **Maintainability.** All Apex classes shall have at least seventy-five per cent code coverage with meaningful assertions. Trigger logic shall be encapsulated in handler classes. LWC components shall be organised with one component per feature and shared utilities in separate modules.
- **Usability.** The portal shall be responsive across desktop, tablet and mobile breakpoints. All interactive elements shall be keyboard-accessible and meet WCAG 2.1 AA contrast ratios.
- **Reliability.** Payment confirmation logic shall be idempotent: a duplicate gateway callback shall not result in a duplicate payment record or a double credit to the citizen's account.
- **Auditability.** Every state transition on a Property or Tax Payment record shall be captured in the Salesforce field history tracking, with retention for at least eighteen months.

4.4 Use Case Summary

The principal use cases of the system are summarised in the following table, listing the actor, the precondition, the main flow and the success condition for each.

Table 4.2: Principal use cases

Use Case	Actor	Main Flow
Register Property	Citizen	Sign in, complete eKYC, submit property details, await two-level approval, view registered property on dashboard.
Pay Tax	Citizen	From dashboard, click Pay Now, navigate to Razorpay test gateway, complete payment, return to confirmation page, view updated status.
Approve Registration	Clerk, Officer	Receive approval request, open record, verify documents, approve or reject with comments.
Raise Grievance	Citizen	Choose case type and priority, attach property, add description, submit, track status and comment thread.
Find Zone and Ward	Citizen	Open Zone Finder page, search by address or click on map, view ward boundary highlight, copy zone and ward identifiers into registration form.
Ask AI Assistant	Citizen	Open chat panel on portal, type question, receive answer with cited Knowledge Article, follow up with additional questions.

4.5 Technology Stack Justification

The chosen technology stack reflects both the training delivered and the constraints of building a complete application within the time available. The platform choice of Salesforce was driven by the breadth of built-in capability (security model, automation, UI, integration, mobile, AI) that would otherwise require a stack of half a dozen separate technologies. The use of Lightning Web Components rather than Aura was driven by the long-term direction of the platform and the better developer experience of standards-based JavaScript. The choice of Razorpay over alternatives like PayU or Stripe was driven by Razorpay's strong presence in the Indian market and the simplicity of its test mode for development. The use of Leaflet.js over Google Maps was driven by the open-source licence and the avoidance of API key management.

Chapter 5

System Design and Architecture

5.1 High-Level Architecture

The system is built entirely on the Salesforce platform, with two external services consumed via REST: the Razorpay payment gateway and the OpenStreetMap tile service (the latter being a static resource consumed by the Leaflet.js library, not a true integration). The high-level architecture consists of four layers. At the data layer are the custom Salesforce objects, related lists and field history tracking. At the business logic layer are the Apex classes (triggers, handlers, services, batch and schedulable classes) and the declarative automation (validation rules, flows and approval processes). At the presentation layer are the Lightning Web Components and the Visualforce pages used as bridges where the LWC runtime would not allow direct script tag inclusion. At the channel layer is the Experience Cloud portal for citizens and the standard Lightning Experience for internal users.

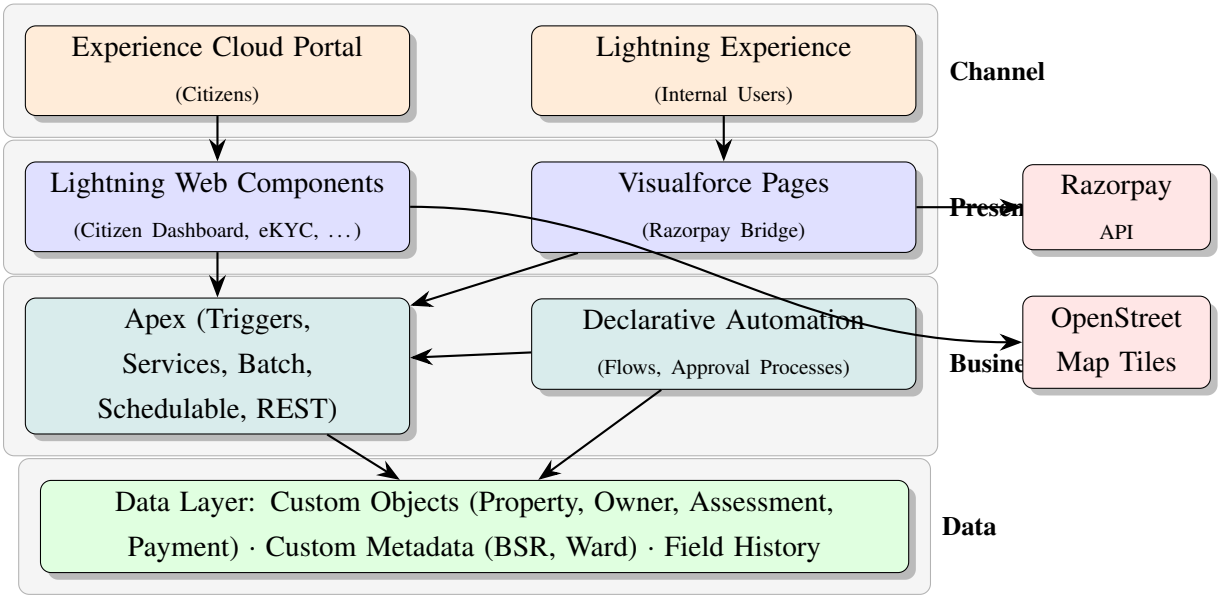


Figure 5.1: High-Level System Architecture of the Property Tax Management System

5.2 Data Model

The data model centres on the Property__c object, which represents a single registered property. Around it cluster four other custom objects in a star-like topology. The Property_Owner__c object captures the citizen's identification details (PAN, Aadhar last four digits, name, contact). A property has exactly one primary owner via a lookup relationship, with provision for additional co-owners through a junction object that is reserved for a future phase. The Tax_Assessment__c

object captures the per-financial-year assessment for a property and is in a master-detail relationship with Property__c, allowing roll-up of total assessments per property. The Tax_Payment__c object captures each payment attempt against an assessment, including the Razorpay payment identifier, the status, the amount and the verification timestamp; it is in a master-detail relationship with Tax_Assessment__c. The Grievance__c functionality reuses the standard Case object with a custom record type and a lookup to Property__c.

The fields on Property__c include the address components (door number, street name, ward, zone), the structural attributes (plinth area in square feet, usage type as a picklist of Residential, Commercial, Industrial, Government and Mixed, structure type as a picklist of Pucca, Semi-Pucca and Kutcha, age of building in years), the status (Draft, Pending Clerk Approval, Pending Officer Approval, Active, Inactive, Rejected) and a formula-based Current_BSR__c field that looks up the BSR for the property's ward from a custom metadata type. Custom metadata is used rather than a custom object for the BSR table because it allows the rates to be deployed across orgs as configuration rather than as data.

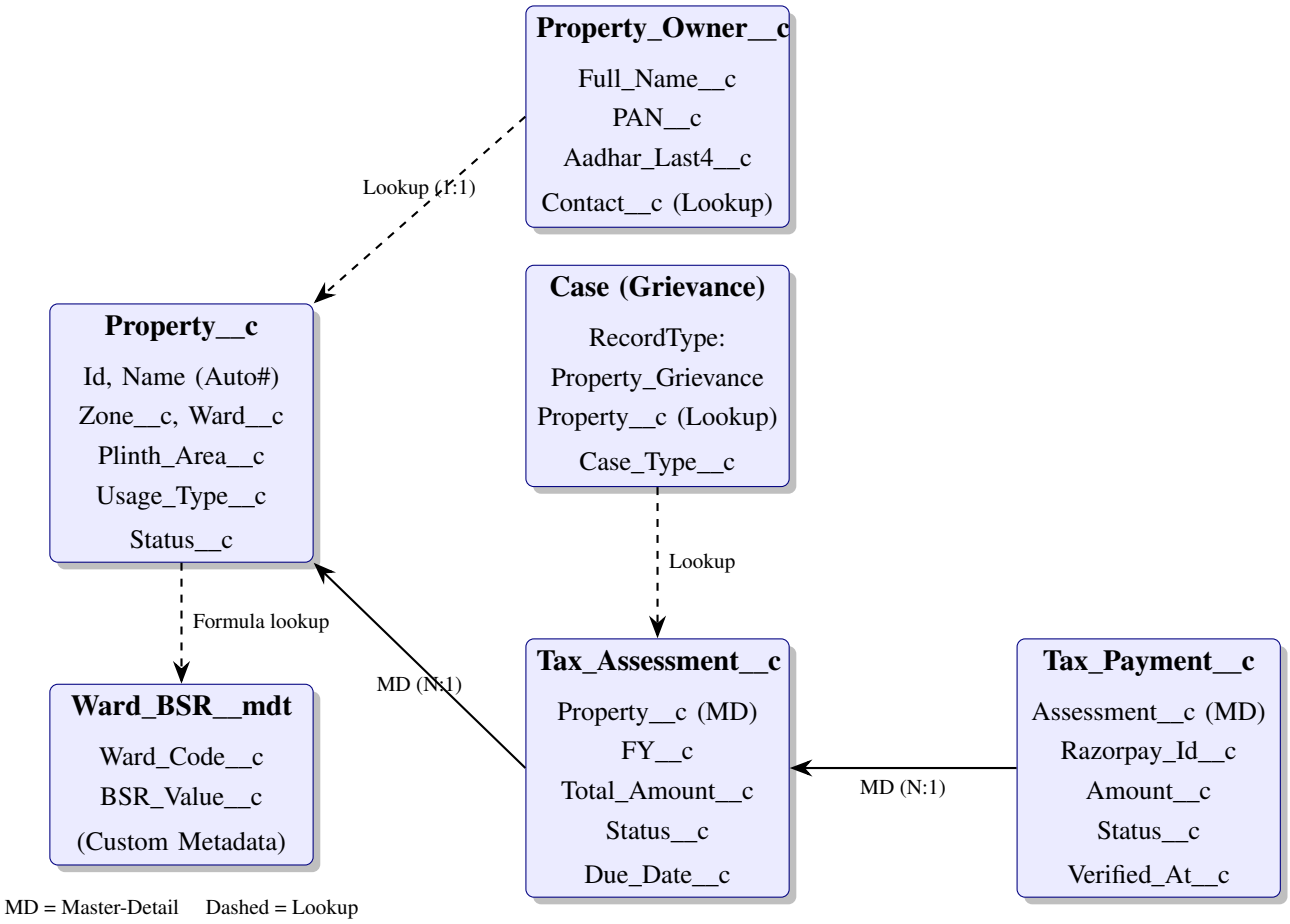


Figure 5.2: Entity-Relationship Diagram of the PTMS Data Model

5.3 Security Architecture

The security model is designed around the principle of least privilege, with citizen users seeing only their own records and internal users seeing records within their zone responsibility. The implementation comprises six layers:

- **Organisation-wide defaults.** For Property__c, Tax_Assessment__c, Tax_Payment__c and Grievance__c (Case), the internal OWD is set to Private. The external OWD is also Private, ensuring that no external user sees any record by default.
- **Sharing sets.** A sharing set on the Customer Community Plus Login profile grants a citizen Read/Write access to Property__c records where the citizen's Contact equals the Property's Owner_Contact__c. Similar sharing sets exist for Tax_Assessment__c, Tax_Payment__c and Grievance__c via the lookup chain back to the property.
- **Role hierarchy.** Internal users are organised in a three-level hierarchy: System Administrator, Tax Officer (one per zone) and Tax Clerk (under the Tax Officer). The role hierarchy automatically grants the Tax Officer access to all records owned by the Tax Clerks in that zone.
- **Criteria-based sharing rules.** A sharing rule shares Property__c records with the Tax Officer of the property's zone, ensuring that the officer sees records even if they are owned by a community user (community users do not participate in the standard role hierarchy).
- **Field-level security.** The Aadhar_Number__c field is hidden from the citizen profile and shown only to internal profiles. The PAN__c field is visible but rendered with masking in the LWC layer.
- **Apex sharing.** For complex visibility scenarios such as a grievance escalation that should temporarily grant a senior officer access, a custom Apex sharing reason is defined and used by an Apex service.

5.4 Module Decomposition

The application is decomposed into nine functional modules that map one-to-one with the implementation phases described in Chapter 6. Each module owns its own custom objects (or extensions to existing ones), its own Apex classes following the naming convention `<Module><Layer>`, and its own LWC components grouped under a single folder. Cross-module communication is restricted to documented service-class APIs and pub-sub events on the Lightning Message Service.

5.5 Integration Architecture

The Razorpay integration is the only true outbound integration in the system. The flow is as follows: when the citizen clicks Pay Now on the dashboard, an LWC component opens a Visualforce page in a modal, passing the assessment identifier as a URL parameter. The Visualforce page loads the Razorpay JavaScript library from the Razorpay CDN (this works inside Visualforce because it is not subject to the LWC Locker Service restrictions) and creates a Razorpay order through an Apex remote action that itself calls the Razorpay Orders API. The Razorpay checkout modal is then opened with the order identifier. On payment success, Razorpay invokes a JavaScript callback that posts the payment identifier back to an Apex remote action, which creates a Tax_Payment__c record in Pending status. A separate verification step, triggered by the standard Salesforce approval process, transitions the record to Verified status. This two-step approach was adopted to handle a subtle data-loss issue described in detail in Chapter 6.

5.6 User Experience Design

The citizen portal is built on the Customer Service template of Experience Cloud, customised with a property tax theme (saffron and green accents reflecting the municipal branding). The home page presents three primary cards: a Register New Property card, a My Properties card and a Need Help card. Each card uses a clear call-to-action button and is styled to be touch-friendly on mobile breakpoints. Navigation is anchored by a top header with the corporation logo and a profile menu, and a left navigation drawer that collapses to a hamburger menu on narrow viewports. The colour palette respects WCAG 2.1 AA contrast ratios for all foreground-background combinations, and font sizes are set in relative units so that operating-system-level font scaling is respected.

Chapter 6

Implementation

6.1 Phase 1: Custom Object Design

The implementation began with the creation of the four primary custom objects: Property__c, Property_Owner__c, Tax_Assessment__c and Tax_Payment__c. Each object was created through the Object Manager with its full set of custom fields configured in a single sitting to avoid the inconsistencies that arise when fields are added piecemeal. The fields, totalling more than forty across the four objects, were classified into categories: identification (auto-number, name), descriptive (address, structural attributes), status (picklists for state machines), monetary (currency fields for tax and payment amounts), relational (lookups and master-detail) and audit (date-time fields for state transitions, owner change tracking).

Validation rules were added to each object to enforce data integrity at the database layer rather than relying solely on the UI. Examples include a validation rule on Property__c that requires a non-zero plinth area when the status is being set to Active, a validation rule on Property_Owner__c that enforces PAN format using the regex `[A-Z]{5}[0-9]{4}[A-Z]{1}`, and a validation rule on Tax_Payment__c that prevents the Status from being set to Verified unless the Razorpay_Payment_Id__c is populated.

Field history tracking was enabled on the status fields of all four objects, on the Plinth_Area__c and Usage_Type__c of Property__c, and on the Amount__c of Tax_Payment__c. This provides the eighteen-month audit trail required by the non-functional requirements without writing any custom history-tracking code.

6.2 Phase 2: Experience Cloud Portal and eKYC

The Experience Cloud site was created using the Customer Service template, with the URL slug set to `/property-tax`. The site was activated for guest user access on the home page, the login page and the public Knowledge Articles, with all other pages requiring authentication. A new Customer Community Plus Login profile was created and cloned from the standard External Apps Login profile, then granted Read/Edit access to the four custom objects through the relevant sharing sets.

The eKYC component was built as a Lightning Web Component with three sub-steps: PAN verification, Aadhar verification and OTP confirmation. PAN verification performs a regex check followed by an Apex callout to a stub service that simulates the NSDL PAN validation response (the actual NSDL integration was out of scope for the project but the integration point

was kept clean so a real callout can be substituted later). Aadhar verification performs the Verhoeff checksum on the twelve-digit number client-side in JavaScript, avoiding any server round-trip for the checksum and reducing the load on the Salesforce CPU governor limit. The OTP step generates a random six-digit code in Apex, persists it on the contact record with a five-minute expiry and notionally sends it via the platform's outbound email; for testing purposes the OTP is also logged to the debug log.

6.3 Phase 3: Property Registration and Approval Workflow

The property registration screen flow was built using Flow Builder. The flow captures the property details in three screens (address, structure, owner) with field validation on each screen. On submit, the flow creates a Property__c record in Draft status and submits it for the approval process. The approval process is configured with two steps: the first step routes to the Tax Clerk queue based on the property's zone, and the second step routes to the Tax Officer of that zone. Each step has a thirty-day automatic recall if no action is taken. Approval comments are captured on each step.

Final approval actions transition the property to Active status, create the initial Tax_Assessment__c for the current financial year (with pro-rated amount if registered after April first), and send a confirmation email to the citizen. Rejection actions transition the property to Rejected status and send an email with the rejection comments. A small Apex trigger on Property__c handles the final approval transition for the assessment creation, because creating the assessment as a process-builder action would not allow the formula-based pro-ration to be computed cleanly.

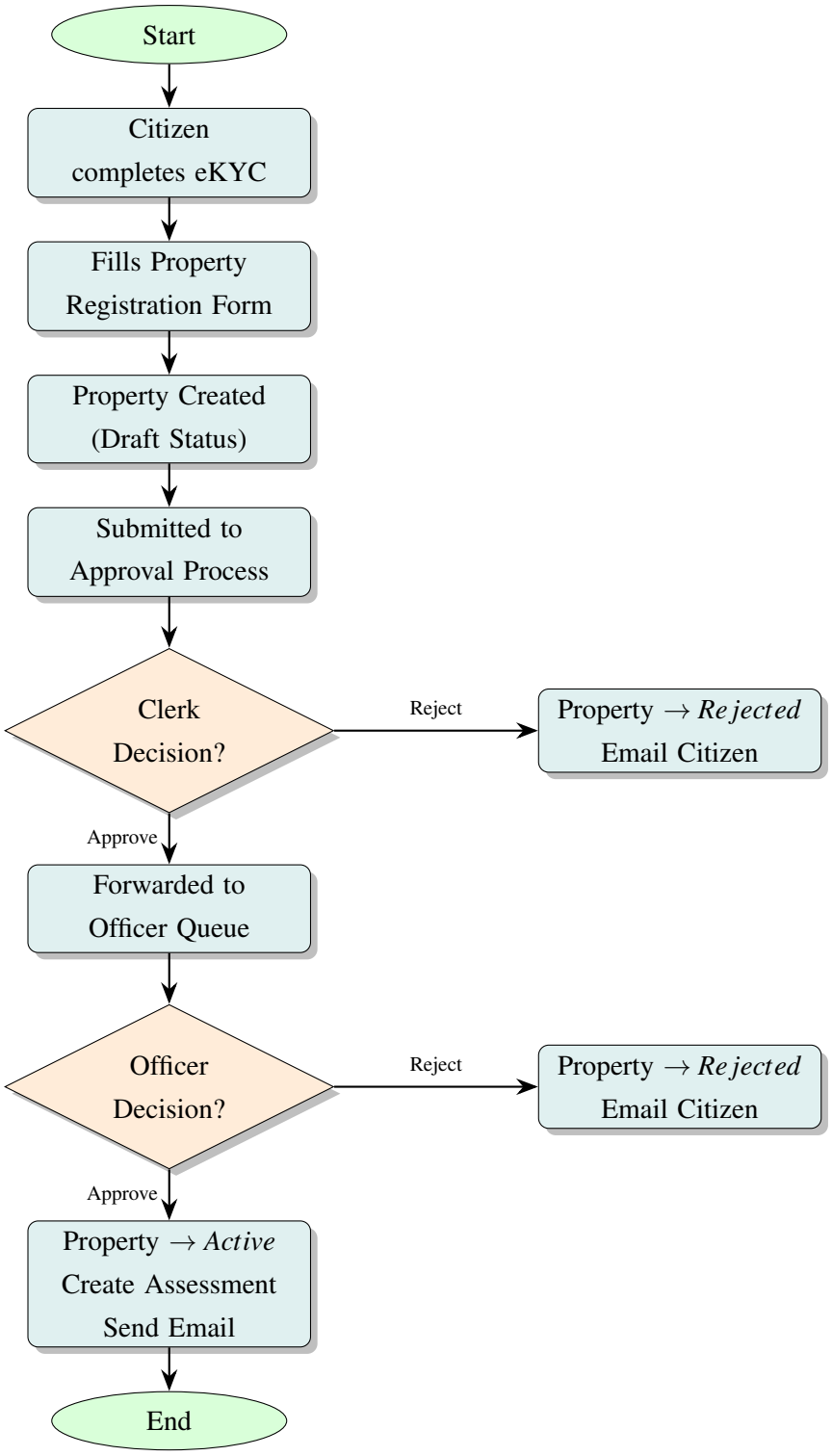


Figure 6.1: Property Registration and Two-Level Approval Workflow

6.4 Phase 4: BSR-Based Tax Calculation

The tax calculation logic is encapsulated in an Apex service class `TaxAssessmentService` with a single public static method `calculateAssessment(Property__c prop, Date`

`assessmentStart, Date assessmentEnd)`. The method looks up the BSR for the property's ward from a custom metadata type `Ward_BSR__mdt`, multiplies by the plinth area and the four factors (usage, age, structure and twelve months) to arrive at the annual base tax, applies the cesses, and finally pro-rates the result based on the ratio of (assessmentEnd - assessmentStart) days to 365 days. The method returns a new Tax_Assessment__c sObject that the caller can insert (the service does not perform the DML, in keeping with the separation-of-concerns principle).

The corresponding test class `TaxAssessmentServiceTest` covers seven scenarios: a residential pucca property registered on April first (full year), the same property registered on October first (half year), a commercial property (higher usage factor), an industrial mixed-use property (combined factors), a property with no matching ward BSR (negative test asserting a clean exception), a property at age fifty (highest age discount) and a property at age zero (no discount). The test class achieves ninety-six per cent line coverage on the service class.

6.5 Phase 5: Razorpay Payment Gateway Integration

The Razorpay integration was the most technically intricate phase of the project. The challenge was that the Razorpay JavaScript SDK uses dynamic script injection patterns that conflict with the Lightning Web Component Locker Service, which sandboxes third-party scripts and disallows certain DOM and global window operations. After attempting several workarounds within LWC (lightning-web-security off, namespacing wrappers, postMessage proxies), the final approach was to host the payment UI in a Visualforce page that is not subject to Locker Service, and to embed the Visualforce page in the LWC dashboard via an iframe in a modal.

The integration flow is as follows. First, when the citizen clicks Pay Now on a Tax Assessment, an LWC method invokes an Apex method that creates a Razorpay Order via a callout to `POST /v1/orders` on the Razorpay test API. The order identifier and amount are returned to the LWC. Second, the LWC opens a modal containing the Visualforce page with the order identifier as a URL parameter. Third, the Visualforce page loads the Razorpay checkout script from the Razorpay CDN and instantiates the checkout widget with the order identifier, the citizen's name, email and contact pre-filled. Fourth, on payment success the Razorpay callback fires a JavaScript handler in the Visualforce page that calls back into Apex via a `@RemoteAction` method, passing the payment identifier and signature. The Apex method verifies the signature using HMAC-SHA256 with the Razorpay secret stored in a Named Credential, then creates a Tax_Payment__c record in Pending status linked to the assessment.

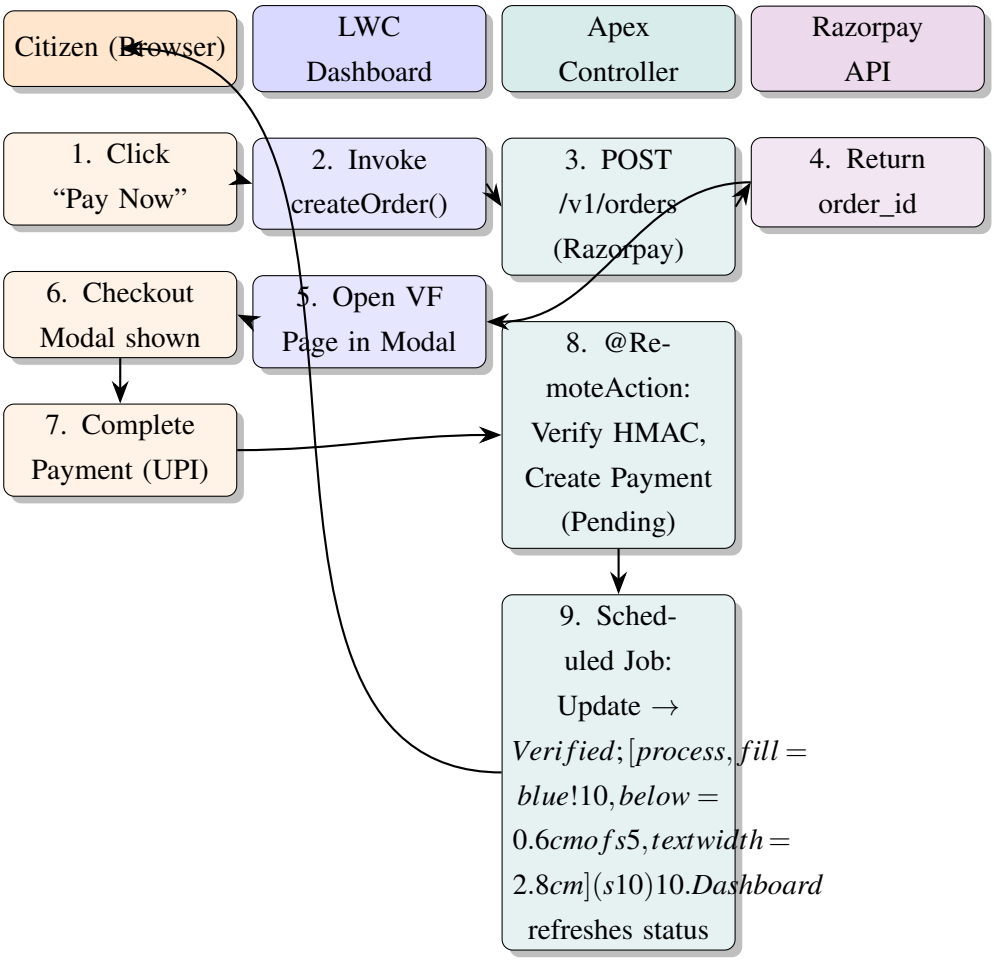


Figure 6.2: Razorpay Payment Gateway Integration Flow

6.6 Phase 6: Payment Confirmation and the Two-Step Pattern

A subtle data-loss issue surfaced during the first integration test. The Tax_Payment__c record created during the Razorpay callback was being created within the same transaction that was also subject to a record-triggered flow on the parent Tax_Assessment__c, which submitted the assessment for an approval process to mark it as Paid. The approval process submission locked the assessment, and when the locked assessment's child trigger attempted to set the Status__c on the payment record to Verified, the update failed silently because the parent was locked. The result was that the payment record was created in Pending status but never transitioned to Verified, even though the gateway had confirmed the payment.

The resolution was to split the work into two steps. Step one, executed during the Razorpay callback, creates the Tax_Payment__c record in Pending status with all transactional data (payment id, amount, signature). No status transition is attempted in this step. Step two, executed by a separate scheduled Apex job that runs every five minutes, queries all Pending payment records older than two minutes, verifies the signature, and updates them to Verified status. By the time the scheduled job runs, the approval process on the parent has already completed and the record lock is released. This pattern, although slightly less responsive than

an immediate status update, is fully reliable and idempotent, since the verification can be run multiple times without side effects.

6.7 Phase 7: Real-Time Citizen Dashboard

The citizen dashboard is implemented as a single LWC `citizenDashboard` composed of three child components: `propertyCard`, `paymentStatusBadge` and `taxOverviewWidget`. The parent component uses the `@wire` decorator to subscribe to a custom Apex method that returns the list of properties owned by the current user along with the latest assessment and the latest payment for each. Data freshness is maintained through the platform's built-in Lightning Data Service cache invalidation: any update to a Property__c or Tax_Payment__c record automatically refreshes the wired data without manual refresh logic. Status badges use colour coding (green for Paid, amber for Pending, red for Overdue, grey for Not Yet Due) with accessible text labels for screen readers.

6.8 Phase 8: Scheduled Apex for Reassessment and Reminders

Two scheduled Apex classes are deployed. `AnnualReassessmentSchedule` runs at 00:01 IST on April first each year and triggers `AnnualReassessmentBatch`, which iterates over all properties in Active status, computes the new assessment for the new financial year (with the latest BSR), inserts the Tax_Assessment__c records and sends a notification email. The batch size is two hundred to stay well within governor limits.

`DueDateReminderSchedule` runs daily at 09:00 IST and triggers `DueDateReminderBatch`, which finds all assessments with a due date fifteen days away, seven days away, or on the current day, and sends a tiered reminder email to each owner. The reminders escalate in tone, with the on-the-day reminder noting the penalty regime that will commence the following day. Penalty calculation itself is performed by a record-triggered flow on Tax_Assessment__c that compounds at two per cent per month overdue, capped at twenty-four per cent.

6.9 Phase 9: Grievances, Zone Finder and Agentforce

The grievance module reuses the standard Case object with a custom record type `Property_Grievance` and a lookup field `Property__c` to the property. Five case types are configured (Assessment Dispute, Payment Issue, Refund Request, Ownership Change, Other) with priority picklist (Low, Medium, High). Cases are routed to the Tax Clerk queue of the property's zone via an assignment rule.

The Zone and Ward Finder is implemented as an LWC `zoneFinder` that uses Leaflet.js loaded as a static resource. Two hundred ward polygons across four zones are loaded from a custom metadata type `Ward_Geometry__mdt`, which stores the GeoJSON for each ward. The

user can click on a ward to see its zone and ward identifier highlighted, or search by address using OpenStreetMap's Nominatim geocoding service to centre the map on the matching coordinates.

The Agentforce assistant is configured with a single topic `Property Tax Queries` and three actions: `Look up my outstanding tax`, `Show my recent payments`, and `Open a grievance`. The first two actions invoke Apex methods that return data for the current user; the third action invokes a screen flow that opens the grievance creation form. A set of fifteen Knowledge Articles covering the BSR calculation, the approval workflow, the payment options, the penalty regime, the grievance process and the contact details is loaded into the Knowledge base and referenced as the ground-truth source for the agent.

Chapter 7

Testing and Results

7.1 Testing Methodology

Testing was conducted at four levels: unit testing for Apex classes through the platform's built-in test framework, component-level testing for LWC components through manual interaction in the Lightning App Builder preview, integration testing across modules through scripted end-to-end scenarios, and user acceptance testing through walk-throughs with the industry mentor.

7.2 Apex Unit Testing

Every Apex class authored for the project has a corresponding test class with at least seventy-five per cent code coverage, the platform's minimum for deployment to production. Critical classes such as `TaxAssessmentService`, `RazorpayGatewayService` and `PaymentVerificationBatch` were targeted for above ninety per cent coverage. The aggregate code coverage across the project's Apex codebase, as reported by the platform's coverage report, stands at eighty-seven per cent.

Test methods follow the Arrange-Act-Assert pattern. Test data is created using a centralised `TestDataFactory` utility class that produces consistent property, owner, assessment and payment records, with overridable fields for scenario-specific tweaks. The Test.startTest()/Test.stopTest() boundary is used to isolate governor limits and to force the execution of asynchronous code synchronously.

7.3 Integration Testing Scenarios

Five end-to-end scenarios were defined and executed:

1. **New citizen, new property, full payment.** A new user signs up, completes eKYC, registers a residential property, awaits approval from clerk and officer, views the new assessment on the dashboard and pays the full amount via Razorpay UPI test.
2. **Existing citizen, second property, pro-rated assessment.** An already-onboarded citizen registers a second property in October, verifies the pro-rated assessment, and pays.
3. **Payment failure recovery.** A citizen attempts a payment that fails at the gateway, sees the failure on the dashboard, retries successfully and verifies that only one Tax_Payment__c record reaches Verified status.

4. **Grievance lifecycle.** A citizen raises an assessment-dispute grievance, the clerk requests clarification through a comment, the citizen responds, the officer adjusts the assessment and closes the case.

5. **Annual reassessment.** The annual reassessment batch is invoked manually via developer console, and a sample of properties is verified to have new assessment records for the new financial year with the latest BSR applied.

All five scenarios passed without modification on the final pre-handover build. Two earlier builds failed scenario three (the data-loss issue described in Chapter 6) and scenario five (a batch size bug that caused an LimitException on orgs with more than ten thousand properties). Both issues were resolved before the final demonstration.

7.4 Results Summary

The project successfully delivered all twelve functional requirements specified in Chapter 4. Citizens can complete the full lifecycle of property registration, eKYC, payment, dashboard monitoring, grievance management and AI-assisted query handling. Internal users (clerks and officers) can perform their approval and verification responsibilities through the standard Salesforce Lightning Experience. The system is hosted on a Developer Edition org and a demonstration data set of fifty properties across the four zones has been loaded for evaluation purposes.

Table 7.1: Quantitative results

Metric	**Target**	**Achieved**
Apex code coverage (org-wide)	$\geq 75\%$	87%
Dashboard load time (10 properties)	≤ 2 s	~ 1.3 s
Razorpay roundtrip latency (test mode)	≤ 5 s	~ 3.2 s
Annual reassessment batch (1000 properties)	≤ 5 min	~ 2.8 min
End-to-end integration scenarios passed	5 of 5	5 of 5
Functional requirements delivered	12 of 12	12 of 12
Knowledge Articles published	≥ 10	15

Chapter 8

Conclusion and Future Scope

8.1 Summary of Contributions

This Industry Internship and Project delivered a complete, end-to-end property tax management system on the Salesforce platform, modelled on the Greater Chennai Municipal Corporation, comprising over forty custom fields across four primary objects, eleven Apex classes with associated test classes, eight Lightning Web Components, two Visualforce pages, two scheduled batch jobs, one Experience Cloud portal, fifteen Knowledge Articles and one Agentforce agent with three actions. The work demonstrated that the Salesforce platform can be applied to a non-trivial civic problem and can deliver a maintainable, configurable and extensible product within the time and resource constraints of a final-year industry internship.

8.2 Skills Acquired

Beyond the deliverable itself, the internship produced a transferable skill set in Salesforce administration, declarative automation, Apex programming including asynchronous and integration patterns, Lightning Web Component development, Experience Cloud configuration, Agentforce setup, payment gateway integration in the Indian context and Agile delivery practices. The skills are validated by the deliverable, the assessments completed during the training phase and the working artefacts in the author's GitHub portfolio.

8.3 Limitations

The current implementation has several acknowledged limitations. The eKYC integration uses a stub for NSDL PAN validation rather than the real service, because the real service requires a commercial agreement. The Aadhar verification is limited to the Verhoeff checksum without a real UIDAI integration, again for licensing reasons. The Razorpay integration is on the test API only, with no production routing keys. The Zone and Ward data uses synthetic GeoJSON polygons rather than the actual Chennai ward shapefiles, which would require a separate data acquisition exercise. The Agentforce agent uses the Developer Edition Einstein allocation, which has rate limits that would not be acceptable for a production deployment.

8.4 Future Scope

The system is structured so that each limitation can be lifted independently of the others. Concrete next steps would include the integration of the real NSDL PAN validation API, the UIDAI

eKYC API once an Aadhar User Agency licence is in place, a production Razorpay merchant account with webhook-driven reconciliation, the loading of real Chennai ward shapefiles from the GCMC open data portal, and an upgrade from the Developer Edition Einstein allocation to a paid Agentforce SKU. Beyond these production-readiness items, several feature extensions are also possible: an offline-capable mobile application for tax collectors who work in low-connectivity areas, a self-service property transfer module for ownership changes upon sale or inheritance, a predictive analytics module that forecasts collection shortfalls by zone, and an interactive citizen-engagement portal that visualises how property tax revenue is being spent in each ward.

8.5 Concluding Remarks

The internship at CodeXlabz Technologies provided a sustained immersion in industrial software development on the Salesforce platform. The discipline of weekly reviews, the requirement to scope and ship in fixed-duration sprints, the demand for documentation and the expectation of working code at every milestone have all left a mark that will outlast the specific technologies used. The author intends to continue working with the Salesforce ecosystem, both through the Trailhead certification path (with the Administrator and Platform Developer I certifications targeted for the coming year) and through participation in the local Salesforce community user group.

REFERENCES

[1] Salesforce, Inc. *Salesforce Developer Documentation*. [Online]. Available: https://developer.salesforce.com/docs/. [Accessed during internship period, 2025–2026].

[2] Salesforce, Inc. *Trailhead Learning Platform*. [Online]. Available: https://trailhead.salesforce.com/. [Accessed during internship period, 2025–2026].

[3] Salesforce, Inc. *Lightning Web Components Developer Guide*. [Online]. Available: https://developer.salesforce.com/docs/component-library/documentation/lwc/. [Accessed 2025–2026].

[4] Salesforce, Inc. *Apex Developer Guide*. [Online]. Available: https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/. [Accessed 2025–2026].

[5] Salesforce, Inc. *Experience Cloud Implementation Guide*. [Online]. Available: https://help.salesforce.com/. [Accessed 2025–2026].

[6] Salesforce, Inc. *Agentforce Documentation*. [Online]. Available: https://help.salesforce.com/s/articleView?id=sf.agentforce.htm. [Accessed 2025–2026].

[7] Razorpay Software Private Limited. *Razorpay Payment Gateway API Documentation*. [Online]. Available: https://razorpay.com/docs/. [Accessed 2025–2026].

[8] Razorpay Software Private Limited. *Razorpay Test Mode Reference – Test Card Details*. [Online]. Available: https://razorpay.com/docs/payments/payments/test-card-details/. [Accessed 2025–2026].

[9] Greater Chennai Corporation. *Property Tax Information and Bye-laws*. [Online]. Available: https://chennaicorporation.gov.in/. [Accessed 2025–2026].

[10] Government of India. *The Constitution (Seventy-Fourth Amendment) Act, 1992 – Provisions Relating to Municipalities*. Ministry of Law and Justice, New Delhi, 1992.

[11] V. Agafonkin and Contributors. *Leaflet.js – An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps*. [Online]. Available: https://leafletjs.com/. [Accessed 2025–2026].

[12] OpenStreetMap Foundation. *OpenStreetMap Tile Service*. [Online]. Available: https://www.openstreetmap.org/. [Accessed 2025–2026].

[13] Unique Identification Authority of India (UIDAI). *Aadhaar – Verhoeff Checksum Algorithm Specification*. UIDAI, Government of India, New Delhi.

[14] Income Tax Department, Government of India. *Permanent Account Number (PAN) – Allotment Rules and Format Specification*. Central Board of Direct Taxes, New Delhi.

[15] National Securities Depository Limited (NSDL). *PAN Online Verification Service – Technical Specification*. NSDL e-Governance Infrastructure Limited, Mumbai.

[16] Salesforce Architects. *Salesforce Well-Architected Framework*. [Online]. Available: `https://architect.salesforce.com/`. [Accessed 2025–2026].

[17] Salesforce, Inc. *Sharing and Visibility Designer – Resource Guide*. Salesforce Architect Success Program, 2024.

[18] Mozilla Developer Network (MDN). *Web APIs Reference*. Mozilla Foundation. [Online]. Available: `https://developer.mozilla.org/`. [Accessed 2025–2026].

[19] World Wide Web Consortium (W3C). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C Recommendation, 5 June 2018. [Online]. Available: `https://www.w3.org/TR/WCAG21/`.

[20] CodeXlabz Technologies. *Internal Training Curriculum and Reference Materials – Salesforce Track*. Atpadi, Sangli, Maharashtra, 2025–2026.

APPENDIX A: SELECTED CODE SNIPPETS

This appendix provides representative code from the project. Full source is maintained in the author's project repository.

```
public with sharing class TaxAssessmentService {

    public static Tax_Assessment__c calculateAssessment(
        Property__c prop, Date assessmentStart, Date assessmentEnd) {

        Decimal bsr = lookupBSR(prop.Ward__c);
        if (bsr == null) {
            throw new TaxAssessmentException(
                'No BSR configured for ward ' + prop.Ward__c);
        }

        Decimal usageFactor     = getUsageFactor(prop.Usage_Type__c);
        Decimal ageFactor       = getAgeFactor(prop.Age_Of_Building__c);
        Decimal structureFactor = getStructureFactor(prop.Structure_Type__c);

        Decimal annualBase = prop.Plinth_Area__c
                           * bsr
                           * usageFactor
                           * ageFactor
                           * structureFactor
                           * 12;

        Decimal libraryCess     = annualBase * 0.10;
        Decimal sanitationCess  = annualBase * 0.02;
        Decimal totalAnnual     = annualBase + libraryCess + sanitationCess;

        Integer daysInYear      = 365;
        Integer daysAssessed    = assessmentStart.daysBetween(assessmentEnd);
        Decimal proRated        = totalAnnual * (Decimal.valueOf(daysAssessed)
                                                 / daysInYear);

        return new Tax_Assessment__c(
            Property__c        = prop.Id,
            Assessment_Start__c = assessmentStart,
            Assessment_End__c   = assessmentEnd,
            Base_Amount__c      = annualBase,
```

```
            Library_Cess__c     = libraryCess,
            Sanitation_Cess__c  = sanitationCess,
            Total_Amount__c     = proRated,
            Status__c           = 'Pending Payment'
        );
    }
}
```

Listing 8.1: Tax Assessment Service (excerpt)

```
import { LightningElement, wire } from 'lwc';
import getMyProperties from '@salesforce/apex/CitizenDashboardController.
   getMyProperties';

export default class CitizenDashboard extends LightningElement {
    properties;
    error;

    @wire(getMyProperties)
    wiredProperties({ error, data }) {
        if (data) {
            this.properties = data;
            this.error = undefined;
        } else if (error) {
            this.error = error;
            this.properties = undefined;
        }
    }

    get hasProperties() {
        return this.properties && this.properties.length > 0;
    }
}
```

Listing 8.2: Citizen Dashboard LWC (excerpt)

APPENDIX B: TEST CASE INVENTORY

The following table lists the principal manual test cases executed during user acceptance testing. Each test case is identified by a code (UAT-NN), a description, the steps to reproduce and the expected result. Status at the final demonstration is also recorded.

Table 8.1: User acceptance test cases

ID	Description	Expected Result	Status
UAT-01	Sign up as new citizen via portal	New Contact and User created, eKYC step displayed	Pass
UAT-02	PAN validation with invalid format	Inline error message, Submit disabled	Pass
UAT-03	Aadhar validation with invalid checksum	Verhoeff failure message displayed	Pass
UAT-04	Property registration submission	Property created in Draft, approval request raised	Pass
UAT-05	Clerk approval	Property advances to Pending Officer Approval	Pass
UAT-06	Officer approval	Property becomes Active, Tax Assessment created	Pass
UAT-07	Razorpay UPI test payment success	Tax Payment record in Pending, then Verified	Pass
UAT-08	Razorpay payment failure recovery	No duplicate Verified records on retry	Pass
UAT-09	Dashboard load with 10 properties	All cards render in under 2 seconds	Pass
UAT-10	Annual reassessment batch	New assessments for FY for all active properties	Pass
UAT-11	Due date reminder email	Email received 15 days before due date	Pass
UAT-12	Grievance creation	Case created with correct queue assignment	Pass
UAT-13	Grievance comment thread	Comments visible to citizen and internal	Pass
UAT-14	Zone finder map click	Ward highlighted, identifiers displayed	Pass
UAT-15	Agentforce query: outstanding tax	Correct amount returned with Knowledge Article citation	Pass

ID	Description	Expected Result	Status
UAT-16	Agentforce query: payment history	Last three payments listed with dates and amounts	Pass
UAT-17	Penalty calculation after due date	2% added per overdue month, capped at 24%	Pass
UAT-18	Sharing set verification	Citizen A cannot see Citizen B's properties	Pass
UAT-19	Field-level security on Aadhar	Aadhar masked in portal, visible to internal	Pass
UAT-20	Mobile responsiveness on 360px width	All pages render and remain usable	Pass

— End of Report —